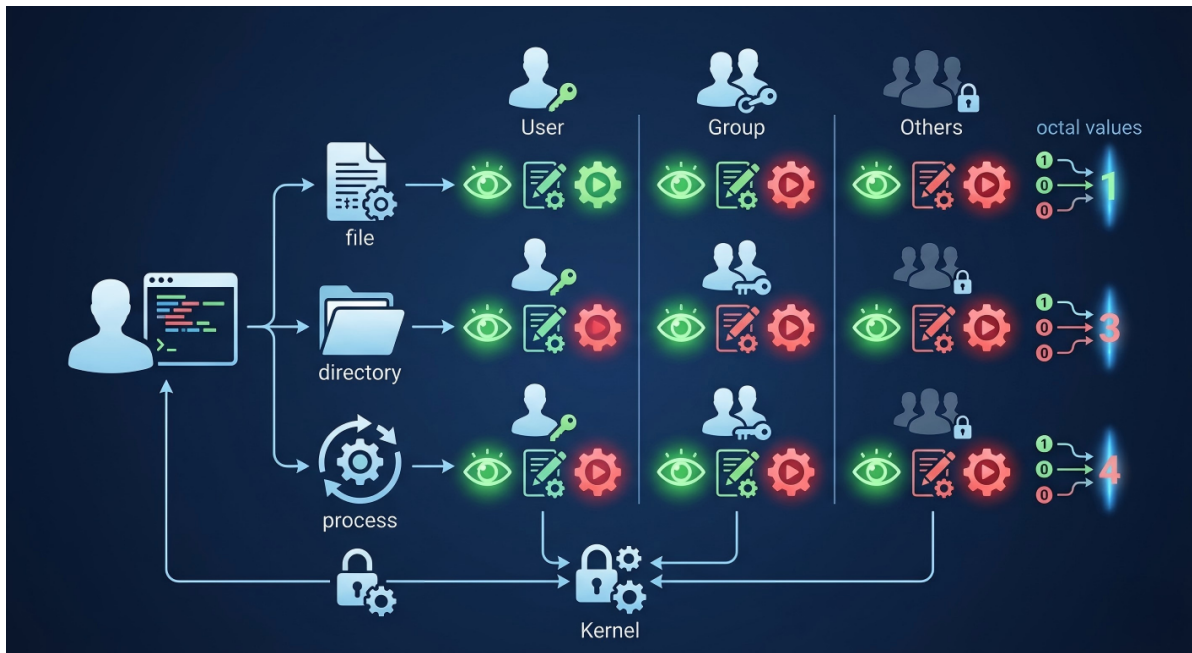


Linux权限管理



一、ACL权限

什么是ACL权限？

ACL 权限是一种允许管理员为文件和目录设置比传统的所有者、组和其他（rwx）权限更精细、更具体的访问控制规则的机制。

比如：有一个项目目录 `/projects/alpha` 。

所有者 (Owner) `project_leader` (项目负责人)权限rwx，

所属组 (Group): `developers` (开发团队)权限rwx，

其他人 (Others) 没有任何权限 (---)，因为公司项目代码闭源

现在，出现了新的需求，传统权限难以优雅解决：

- 1.市场部的 `marketing_user` 需要临时查看 `/projects/alpha` 目录下的某些报告（只读），但她不是 `developers` 组的成员，也不应该是该项目的负责人。
- 2.实习生 `intern_user` 需要对 `/projects/alpha/uploads` 这个子目录有读写权限，以便上传文件，但她不应该有权限修改 `/projects/alpha` 目录下的其他内容。
- 3.qa_team (测试团队) 需要对整个 `/projects/alpha` 目录有只读和执行权限，以便他们可以浏览和测试项目，但他们不属于 `developers` 组

传统权限的困境：

- 1.你不能把 `marketing_user` 加到 `developers` 组，因为那样她会获得对所有 `developers` 组有权限文件的写权限，这超出了需求
- 2.你不能把 "其他人" (Others) 的权限设置为只读，因为这可能让公司里所有不相关的其他人也能看到项目内容，不安全。
- 3.同样，将 `intern_user` 加入 `developers` 组权限过大，`/projects/alpha/uploads` 设置 "其他人" 权限为 `rwx` 过于危险
- 4.一个文件或目录只能有一个主用户组。你不能同时让它属于 `developers` 和 `qa_team`

综上所述，传统权限无法解决，但是使用 ACL，可以为这些特定用户和组精确地授予所需的权限，而不影响项目目录原有的传统权限。

因此正常情况下，安全/运维对Linux操作系统的权限分配都是采用ACL权限进行管理，而不是依靠传统目录权限，通常是重要敏感文件/目录的所属人和所有者都是超级管理员（如root），其他人没有任何权限，然后如果有其他用户有访问需要，则使用ACL权限额外添加上。这也符合最小化权限的安全规范

(1.1) ACL权限的支持

1.1.1 如何确认你的磁盘分区是否支持ACL权限

如果磁盘分区不支持ACL权限，是无法使用ACL对文件/目录进行权限管理的

centos7.x及以上的发行业本，默认使用 XFS文件系统。对于 XFS 文件系统，默认就是支持acl权限的。可以使用下面两种方式进行确认：

方式一：查看磁盘的详细信息

```
# 先查看磁盘的文件系统类型
df -Th
# 如果是centos7以下版本，为 ext2、ext3 或 ext4 类型的
# dumpe2fs /dev/sda1
# centos7以上版本则为xfs类型
xfs_info /dev/sda1
```

```
[root@theon ~]# xfs_info /dev/sda1
meta-data=/dev/sda1            isize=512    agcount=4, agsize=32000 blks
                         =      sectsz=512    attr=2, projid32bit=1
                         =      crc=1        finobt=0 spinodes=0
data      =                    bsize=4096    blocks=128000, imaxpct=25
                         =      sunit=0      swidth=0 blks
naming    =version 2           bsize=4096    ascii-ci=0 ftype=1
log       =internal            bsize=4096    blocks=855, version=2
                         =      sectsz=512    sunit=0 blks, lazy-count=1
realtime  =none                extsz=4096    blocks=0, rtextents=0
```

如上图，如果attr=2，说明此磁盘分区 XFS 文件系统具有支持扩展属性，支持 ACL权限

方式二：查看挂载信息

```
# 直接查看磁盘分区的mount相关选项信息即可
mount | grep /dev/sda1
```

```
[root@theon ~]# mount | grep /dev/sda1
/dev/sda1 on /boot type xfs (rw,relatime,seclabel,attr2,inode64,noquota)
[root@theon ~]#
```

如上图，如果磁盘分区的挂载选项（括号里面的内容）里面没有noacl就说明支持ACL权限

1.1.2 如何开启ACL权限支持（ext3、ext4）

较老的操作系统（ext3、ext4 文件系统）可以通过mount命令重新挂载磁盘分区，并添加上acl参数的形式进行开启即可

```
# 临时开启, 只在 ext3、ext4 等ext文件系统上支持
mount -o remount,acl /dev/sda1
# 永久开启
# 1、写入/etc/fstab, 例如: UUID=844cdf26-b25e-4b14-b83d-cdbc9b539c1a / xfs
defaults,acl 0 0 然后执行mount -o remount /
# 2、写入到/etc/rc.local, 例如: echo "mount -o remount,acl /dev/sda1" >>
/etc/rc.local && chmod +x /etc/rc.local
```

centos7 及以上的系统不需要额外操作开启

(1.2) ACL权限管理

使用ACL权限管理之前, 如果是最小化安装, 需要安装ACL管理包

```
yum -y install acl
```

1.2.1 查看ACL权限

命令格式: `getfacl [目录/文件名]`

范例: `getfacl 456.txt`

```
getfacl /opt
```

没有设置过ACL权限的输出信息

```
[root@theon ~]# getfacl 456.txt
# file: 456.txt
# owner: root
# group: root
user::rw-
group::r--
other::r--

[root@theon ~]# getfacl /opt
getfacl: Removing leading '/' from absolute path names
# file: opt
# owner: root
# group: root
user::rwx
group::r-x
other::r-x

[root@theon ~]#
```

为保持getfacl输出信息格式, 如果是绝对路径的目录, 会在显示时自动去掉开头的 / 号

1.2.2 设置ACL权限

命令格式: `setfacl [选项] [选项2]:[用户/组]:[用户名/组名]:[权限] [选项3] [目录/文件名]`

常见命令: `setfacl -m d:u:wduser:rwx -R [目录]`

```
setfacl -m u:wduser:rwx [目录]/[文件]
```

[选项]:

-m 设定ACL权限

-x 删除ACL权限

-k 删除默认ACL权限

[选项2]:

d 设置默认ACL权限 (针对目录)

[选项3]:

-R 递归设定ACL权限 (针对目录)

[用户/组]:

u 表示给用户添加ACL权限

g 表示给组添加ACL权限

[用户名/组名]:

如果是给用户添加ACL权限, 则是用户名, 如果给组添加ACL权限, 则是组名

[权限]:

使用rwx 或者 数字 都可以

范例:

```
setfacl -m u:bike:rx /opt/test #给test目录添加一个用户bike, 权限为rx读和执行, 用户不属于所有者所属组和其他人, 需要注意用户bike必须存在
setfacl -m g:bike:6 /opt/test #给test目录添加一个组bike, 权限为rx读和执行, 组不属于所有者所属组和其他人, 需要注意组bike必须存在
setfacl -m d:u:bike:rx /opt/test #给test目录添加一个默认权限, 当次目录下新建目录和文件时, 自动添加一个用户bike, 权限为rx读和执行, 用户不属于所有者所属组和其他人, 需要注意的是, 目录test本身不受影响
setfacl -m u:bike:rx -R /opt/test #给test目录以及目录下的所有文件、子目录添加一个用户bike, 权限为rx读和执行, 用户不属于所有者所属组和其他人, 需要注意用户bike必须存在
```

扩展: 设置ACL权限后的信息输出

```
getfacl /opt/test
```

```
[root@theon opt]# getfacl /opt/test
getfacl: Removing leading '/' from absolute path names
# file: opt/test
# owner: root
# group: root
user::rwx
user:bike:r-x
group::r-x
mask::r-x
other::r-x

[root@theon opt]#
```

如上图, 设置了acl权限后, 可以看到目录多出一个用户bike, 有读和执行权限, 多出一个mask属性, mask是用来指定最大有效权限的。如果给用户赋予了ACL权限, 还需要和mask的权限"相与"才能得到用户的真正权限。默认情况下mask属性是不影响你设置的acl权限的

1.2.3 删除ACL权限

删除用户/组的ACL权限

命令格式: `setfacl -x [用户/组]:[用户名/组名] [目录/文件名]`

范例: `setfacl -x u:bike /opt/test`

删除默认权限

命令格式: `setfacl -k [目录/文件名]`

范例: `setfacl -k /opt/test`

二、文件特殊权限

下面介绍三种特殊的文件权限，这些权限可分配对象不一样，有的权限只能分配给文件，有的只能给目录，有的两种都可以，但是效果不一样。

(2.1) SUID权限

2.1.1 介绍

全称是SetUID权限，只能分配给文件，SUID权限特性如下

- 前提：只有可以执行的程序/二进制文件才能设定SUID
- 前提：权限命令执行者要对该程序拥有x(执行)权限，通常来说就是文件的其他人拥有x执行权限，如/usr/bin下的命令
- 命令执行者在执行该程序/二进制文件时获得该程序文件属主的身份(在执行程序的过程中灵魂附体为文件的所有者)
- SetUID权限只在该程序执行过程中有效，也就是说身份改变只在程序执行过程中有效

权限使用范例：（在所有者的X权限变为了S权限，即为SetUID权限）

```
[root@theon opt]# ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root root 27832 Jun 10 2014 /usr/bin/passwd
[root@theon opt]#
```

最常见的就是/usr/bin/passwd 命令了，因为passwd命令修改密码时，会修改/etc/shadow 文件。当普通用户执行passwd命令时，会临时提升为passwd命令的所有者root，以root的身份去修改/etc/shadow 文件，最终完成密码修改。

相对应的cat命令，没有SUID权限，普通用户就不能去查看/etc/shadow 文件

2.1.2 设置SUID权限

添加权限：使用chmod命令进行设置即可

```
chmod 4755 文件 #最前面的4表示SUID权限
chmod u+s 文件
```

演示：我们尝试给cat命令设置SUID权限，看看普通用户是否就可以查看/etc/shadow 文件了

原理：普通用户执行cat命令时，会“灵魂附体”，变成cat命令的所有者root，root是可以查看/etc/shadow 文件的，因此赋予cat命令SUID命令后，普通用户就可以查看/etc/shadow 文件了【原则上，所有文件都可以查看了？】

删除权限：还是使用chmod命令进行设置

```
chmod 755 文件
chmod u-s 文件
```

2.1.3 危险的SUID

由于SUID文件在执行时，权限会提高（权限由普通用户提升为root用户），因此如果某个命令或者可执行程序同时满足如下两个条件：

- 1.命令/程序能进入shell交互页面
- 2.命令/程序被赋予SUID权限

那么，就可以利用这个命令或者可执行程序完成一次提权操作。原理是，普通用户执行命令/可执行程序时，由于SUID权限临时变成root用户，而执行命令/可执行程序又进入了shell交互页面并没有执行完成，那么普通用户就一直是root用户，并且以root用户的身份在shell交互界面中执行命令。

哪些命令可以进入shell交互页面？

<https://gtfobins.github.io/> 中有shell的标签的

	SUID	Sudo	Limited SUID				
gcc	Shell	File write	File read	Sudo			
gcloud	Shell	Sudo					
gcore	File read	SUID	Sudo				
gdb	Shell	Reverse shell	File upload	File download	File write	File read	Library load
	SUID	Sudo	Capabilities				
gem	Shell	Sudo					
genie	Shell	SUID	Sudo				
genisoimage	File read	SUID	Sudo				
ghc	Shell	Sudo					
ghci	Shell	Sudo					

什么是shell交互页面？

就是执行了 /bin/sh 、 /bin/bash 这些shell环境命令

我们以find命令为例子

 **find**  Star 11,576

Shell File write SUID Sudo

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
find . -exec /bin/sh \; -quit
```

```
find . -exec /bin/sh -p \; -quit
```

提权演示：使用find提权

(2.2) SGID权限

2.2.1 介绍

全称是SetGID权限，能分配给文件和目录，SGID权限特性如下

如果分配给文件

- 和SUID类似，区别在于命令执行者在执行该程序时组身份升级为文件的所有组（SUID是普通用户变成root，SGID是普通用户加入了root组）

如果分配给目录

- 普通用户必须对此目录拥有和x权限，才能进入此目录
- 普通用户在此目录中的有效组会变成此目录的属组
- 若普通用户对此目录拥有w权限时，新建的文件的默认属组是这个目录的属组

2.2.2 设置SGID权限

添加权限：使用chmod命令进行设置即可

```
chmod 2755 文件/目录 #最前面的2表示SGID权限
chmod g+s 文件/目录
```

演示：我们使用root在/opt下新建一个目录sgid 然后赋予SGID权限，并给其他人赋予w权限后，使用普通用户进入目录新建文件看看所属组，然后取消SGID权限后，使用普通用户在新建一个文件，看看所属组，并对比两个文件的所属组

删除权限：还是使用chmod命令进行设置

```
chmod 755 文件/目录
chmod g-s 文件/目录
```

(2.3) SBIT权限

2.3.1 介绍

全称是Sticky BIT权限，也叫粘着位权限，只能分配给目录，SBIT权限特性如下

- 普通用户对该目录拥有w和x权限，即普通用户可以在此目录拥有写入权限
- 如果没有粘着位权限，因为普通用户拥有w权限:所以可以删除此目录下所有文件，包括其他用户建立的文件。一旦赋予了粘着位，除了root可以删除所有文件，普通用户就算拥有w权限，也只能删除自己建立的文件，但是不能删除其他用户建立的文件

2.3.2 设置SBIT权限

添加权限：使用chmod命令进行设置即可

```
chmod 1755 目录 #最前面的1表示SBIT权限
chmod o+t 目录
```


演示：我们使用root在/opt下新建一个目录sbit 然后赋予SBIT和777权限,并使用root在目录sbit下创建一个文件，然后授权为777，使用普通用户进入目录新建文件，然后尝试删除

```
# root
mkdir /opt/sbit
chmod 1777 /opt/sbit
echo "root" > /opt/sbit/root.txt
chmod 777 /opt/sbit/root.txt
# 普通用户
cd /opt/sbit
touch bike.txt
rm -f root.txt
rm -f bike.txt
```

删除权限：还是使用chmod命令进行设置

```
chmod 755 目录
chmod o-t 目录
```

其他

播客标题：Linux权限灵魂附体

播客内容：深入探讨了Linux特殊权限。

播客地址：

- 喜马拉雅：<https://www.ximalaya.com/album/118679212>
- 小宇宙：<https://namecard.xiaoyuzhoufm.com/rjrtv>

目的：辅助掌握今天的知识内容，没有时间学习的同学或者已经有基础的同学可以通过播客快速掌握大致内容

作业

一、安全情景任务

还记得ACL权限的那个例子吗？

情景描述：某企业有一个项目目录 `/projects/alpha`

所有者 (Owner) `project_leader` (项目负责人)权限rwx，

所属组 (Group): `developers` (开发团队)权限rwx，

其他人 (Others) 没有任何权限 (`---`)，因为公司项目代码闭源

现在，出现了新的需求：

- 1.市场部的 `marketing_user` 需要临时查看 `/projects/alpha` 目录下的某些报告（只读）
- 2.实习生 `intern_user` 需要对 `/projects/alpha/uploads` 这个子目录有读写权限，以便上传文件，但她不应该有权修改 `/projects/alpha` 目录下的其他内容。

- 3. qa_team (测试团队) 需要对整个 /projects/alpha 目录有只读和执行权限，以便他们可以浏览和测试项目。

任务一：复现场景

- 添加五个用户 project_leader、developers、marketing_user、intern_user、qa_team
- 创建目录/projects/alpha和/projects/alpha/uploads
- 更改目录/projects/alpha和其子目录的所有者为 project_leader、所属组为 developers
- 更改目录/projects/alpha和其子目录的权限为rwxrwx---

任务二：使用ACL完成企业需求

- 对项目目录 /projects/alpha 增加一个 marketing_user 用户的r权限
- 对项目子目录 /projects/alpha/uploads 增加一个 intern_user 用户的rw权限
- 对项目目录 /projects/alpha 增加一个 qa_team 组的rx权限

提交内容：

- 执行 getfacl /projects/alpha 后的内容输出截图
- 执行 getfacl /projects/alpha/uploads 后的内容输出截图